Broker Operations Exception Report Generator - Project Plan

Last updated: 2026-05-09

Purpose

Build a clean, static, recruiter-safe Python demo that reads broker-style order/platform event logs and produces:

- a machine-readable shift summary JSON;
- by-symbol trading statistics CSV;
- an order exception log CSV;
- a human-readable broker operations shift report Markdown file;
- optionally, an Excel workbook if it stays easy and dependency-safe.

The project must be framed as a broker operations reporting demo. It is not a trading strategy, not a trading bot, not MT4/MT5 integration, not a live monitor, and not proof of broker administrator access.

Current State

[x] Workspace inspected: `/Users/chris/Documents/New project`.

[x] Git exists with locked checkpoint commits.

[x] Existing local projects searched for reusable patterns.

[x] GitHub account `quantfin33` searched for candidate repos.

[x] Decision made: write clean original code in this workspace.

[x] Reuse patterns only, not live trading/scanner code.

[x] Guided Build Cycle operating system planned.

[x] GBC audit/checkup documentation added.

[x] Repo-level `AGENTS.md` added for future Codex sessions.

[x] Git hygiene ignore rules added for OS files, caches, virtualenvs, temp files, logs, and local env files.

[x] Python project scaffold created.

[x] Sample broker-style CSV data created.

[x] CSV schema validation implemented.

[x] Phase 3 core exception pipeline complete.

[x] Phase 3C broker exception type detection implemented.

[x] Phase 3D severity classification implemented.

[x] Phase 3E recommended action mapping implemented.

[x] Phase 4A exception log output locked.

[x] Phase 4B shift summary JSON output locked.

[ ] Later exception/reporting enrichments implemented.

[ ] Reports generated.

[ ] Tests added and passing.

[ ] README and recruiter-facing project framing written.

[x] First git commit created.

Locked commits:

[x] `de4801e` - Initialize broker ops report scaffold.

[x] `6922829` - Add static broker ops demo data.

Reuse Decision

Use these local projects as references only:

- `/Users/chris/Desktop/mcp_market_intelligence_demo`: best reference for static CSV inputs, CLI commands, validation, Markdown reports, safety language, and tests.
- `/Users/chris/Desktop/crypto_prob_forecast`: reference for data-audit discipline and monitoring vocabulary, not direct code reuse because it includes live/monitoring/Telegram/Binance-adjacent surfaces.
- `/Users/chris/Project1/monte carlo clean version 3 explanation`: reference for audit reports, safe JSON, and project documentation style.
- `/Users/chris/Desktop/public_repo_review/monte-carlo-real-estate-demo`: reference for portfolio framing and README structure.

Do not reuse:

- `binancetracker` live Binance/Telegram code.
- `tradingview-mcp` live/MCP/fork code.
- any account/session/config/log files.
- any private credential, `.env`, browser, app-support, or cache material.

Professional AI-Coding Workflow

For this project, use the same operating habits recommended for professional coding agents:

- Keep a short repo-level instruction file, such as `AGENTS.md`, once implementation starts.
- Keep this plan as a living tracker, similar to a `PLANS.md` file.
- Break the work into small, verifiable slices.

- Make the agent inspect before editing.
- Keep generated outputs reproducible from source inputs.
- Run tests after each meaningful implementation slice.
- Keep domain claims conservative and explicit.
- Review diffs before committing.
- Use AI for implementation speed, but use tests, source citations, and human review for trust.

Guided Build Cycle Working System

GBC means Guided Build Cycle. It is the local gated operating system for building this project with Codex while keeping the scope honest, testable, and auditable.

The cycle is:

```
task brief -> read-only inspection -> approved plan -> bounded implementation -> tests -> audit -> checkpoint
```

Use this cycle for every meaningful project slice. A slice should be small enough that a reviewer can understand the inputs, changed files, expected outputs, and verification evidence in one pass.

GBC Gates

- [x] Gate 0 - Scope Safety: confirm the task is static/demo-only, with no MT4/MT5 connection, broker API, trading bot, live account, credential use, or real execution claim.
- [x] Gate 1 - Inspection: read current files, schemas, sample data, tests, and outputs before planning. Hard stop if required inputs or column contracts are missing.
- [x] Gate 2 - Approved Plan: define exact files, inputs, outputs, rules, tests, and acceptance criteria before editing.
- [ ] Gate 3 - Implementation: make only the approved scoped change. No unrelated refactors or hidden feature expansion.
- [ ] Gate 4 - Verification: run relevant tests, generate reports, confirm output files exist, and validate count tie-outs.
- [ ] Gate 5 - Audit: review diff, safety language, data contracts, exception-rule correctness, generated artifacts, and portfolio honesty.
- [ ] Gate 6 - Checkpoint: record what changed, what passed, what failed, open risks, and the next recommended task.

GBC Artifacts

Keep lean artifacts under `docs/gbc/`:

- `GBC_WORKING_SYSTEM.md`: the operating rules, gates, halt conditions, and artifact expectations.
- `CHECKUP_MATRIX.md`: the domain, schema, rule, output, safety, and git checkups to run.
- `AUDIT_LEDGER.md`: append-only human-readable record of completed GBC cycles.
- `templates/TASK_BRIEF.md`: one task, goal, scope, and acceptance criteria.

- `templates/APPROVED_PLAN.md`: exact implementation plan approved before edits.
- `templates/IMPLEMENTATION_REPORT.md`: files changed, behavior added, and important notes.
- `templates/TEST_REPORT.md`: commands run, results, artifacts checked, and gaps.
- `templates/CHECKPOINT_NOTE.md`: current status, open risks, and next action.

Halt Conditions

Stop and clarify before coding if any of these occur:

- Required CSV inputs, columns, or output contracts are missing or ambiguous.
- A task implies live trading, live broker access, MT4/MT5 Manager API use, credentials, or account-specific data.
- Exception semantics are unclear enough that test expectations would be guessed.
- A requested change would alter protected public outputs without an approved plan.
- Test results, row counts, or generated report totals do not tie out.
- The git diff includes unrelated or user-owned changes that would be mixed into the task.

Audit And Checkup System

The audit system is local and gated. It does not auto-merge, auto-deploy, or give an agent authority to publish changes without review.

- Domain audit: order statuses, bridge statuses, latency thresholds, rejection reasons, market-event awareness, and handover notes stay broker-ops accurate.
- Schema audit: required columns, UTC timestamps, duplicate IDs, null handling, numeric fields, and market-event links are validated.
- Rule audit: every exception rule has a fixture and a test case.
- Output audit: JSON summary, by-symbol CSV, exception log CSV, Markdown report, and optional Excel workbook are internally consistent.
- Safety audit: no secrets, copied credential files, live trading integration, or misleading MT4/MT5/admin claims.
- Git audit: inspect status and diff before commit, keep unrelated files out, and preserve user changes.

Recurring Checkups

- Per task: create a task brief, inspect current state, write an approved plan, then implement only that slice.
- Per phase: update this checklist, regenerate affected artifacts, and record a checkpoint note.
- Pre-commit: run tests, regenerate outputs, inspect git diff, scan for secrets/live integrations, and confirm README/non-claims language.
- Release readiness: verify acceptance criteria, generated sample outputs, portfolio framing, and source/research references.

Build Path Checklist

Phase 0 - GBC Operating System

- [x] Add `AGENTS.md` with project-specific Codex operating rules.
- [x] Add `docs/gbc/GBC_WORKING_SYSTEM.md`.
- [x] Add `docs/gbc/CHECKUP_MATRIX.md`.
- [x] Add `docs/gbc/AUDIT_LEDGER.md`.
- [x] Add lean GBC templates.
- [x] Add `.gitignore` for local-only artifacts.
- [x] Regenerate this project plan PDF after the GBC upgrade.

Phase 1 - Project Scaffold

- [x] Create `pyproject.toml` with Python 3.11+ metadata.
- [x] Create package under `src/broker_ops_report/`.
- [x] Create CLI entrypoint with commands:
  - `run-demo`
  - `validate-inputs`
  - `generate-reports`
- [x] Create `data/`, `outputs/`, and `tests/` folders.
- [x] Add `AGENTS.md` with project-specific instructions for future AI coding sessions.
- [x] Add minimal Phase 1 CLI tests for imports, help output, placeholder exits, and no live dependency requirement.

Phase 2 - Static Demo Data

- [x] Create `data/sample_order_events.csv`.
- [x] Create `data/sample_market_events.csv`.
- [x] Create `docs/DATA_SCHEMA.md`.
- [x] Include realistic broker-style fields: timestamps, order IDs, status, bridge status, route, liquidity provider, latency, slippage, and sample P&L.
- [x] Include intentionally imperfect rows for testing:
  - received but not transmitted;
  - transmitted but no final status;
  - rejected without reason;
  - failed/disconnected bridge;
  - duplicate order ID;
  - missing required field;
  - high-latency event;

- unresolved pending order.

[x] Include abnormal symbol activity and market-event overlap candidates.

[x] Add fixture-only tests for CSV existence, headers, scenario rows, secret scan, and no live dependency.

Phase 3 - Core Validation And Exception Pipeline

[x] Validate required columns and missing core fields.

[x] Validate allowed `status` and `bridge_status` values.

[x] Validate UTC-style timestamp fields when present.

[x] Defer timestamp normalization as optional future validation enhancement.

[x] Validate numeric fields when present.

[x] Validate market-event required columns and related symbols.

[x] Scan input fixtures for obvious secret/live credential patterns.

[x] Validate lifecycle consistency.

[x] Detect duplicate `client_order_id` and `server_order_id`.

[x] Treat duplicate IDs and missing required fields as validation failures, not Phase 3C exception-report rows.

[x] Detect Phase 3C broker exception types:

- received not transmitted;
- transmitted no final status;
- rejected without reason;
- failed bridge status;
- high latency;
- pending follow-up.

[x] Defer abnormal symbol activity to later by-symbol statistics work.

[x] Defer market-event overlap to later enrichment work.

[x] Assign severity: `Critical`, `Warning`, or `Info`.

[x] Attach recommended action for each exception.

Phase 4 - Report Outputs

[x] Generate `outputs/broker_ops_shift_summary.json`.

[ ] Generate `outputs/by_symbol_trading_stats.csv`.

[x] Generate `outputs/order_exception_log.csv`.

[ ] Generate `outputs/broker_ops_shift_report.md`.

[ ] Optionally generate `outputs/broker_ops_shift_report.xlsx` with summary, symbol stats, exceptions, and events sheets.

Phase 5 - Tests And Verification

[ ] Add unit tests for each exception rule.

[ ] Add tests for summary counts.

[ ] Add tests for by-symbol abnormal activity thresholds.

[ ] Add tests for invalid/missing input fields.

[ ] Add tests that generated reports include non-claims and shift handover sections.

[ ] Run the full test suite.

[ ] Run the CLI against the sample data and confirm output artifacts exist.

Phase 6 - README And Portfolio Framing

[ ] Write a concise README with:

- project purpose;
- input/output examples;
- how to run locally;
- sample screenshots or report excerpts if useful;
- limitations and non-claims;
- connection to broker operations, reporting, risk review, and shift handover.

[ ] Make clear that the project does not connect to MT4/MT5, FIX venues, broker APIs, or live accounts.

[ ] Include source/research references in a short documentation section.

Phase 7 - Optional Enhancements

[ ] Excel workbook output.

[ ] HTML report preview.

[ ] Static dashboard only if needed later.

[ ] GitHub Actions test workflow after the first stable commit.

Acceptance Criteria

The v1 project is complete when:

- `python` CLI can generate all required outputs from static sample CSVs.
- Tests pass.
- The report clearly ranks exceptions by severity.
- The shift handover section lists unresolved operational items.
- The README explains the project honestly and avoids production/broker-access claims.
- No live trading, exchange, broker, Telegram, browser-session, or credential files are used.
- GBC artifacts exist and explain how to run task briefs, approved plans, implementation reports, test reports, audits, and checkpoints.

Research Notes For AI-Coding Workflow

The plan follows current best-practice themes from OpenAI Codex, GitHub Copilot, and Claude Code documentation:

- Coding agents work best with clear repository instructions and documented build/test commands.
- Long-running or multi-step work benefits from a living plan file that records current state, completed work, remaining work, and stopping points.
- Tasks should be small enough to review and verify.
- Agents should be asked to run tests, explain changes, and provide evidence of verification.
- Human review remains required before trusting or publishing results.
- The strongest local reference is `/Users/chris/Desktop/crypto_prob_forecast/docs/gbc_prompts.md`, which uses inspect-first prompts, hard-stop conditions, approved plans, implementation reports, and verification summaries.
- The second local reference is `/Users/chris/Desktop/crypto_prob_forecast/docs/CODEX_SUPERVISED_MULTI_AGENT_WORKFLOW.md`, which favors one primary Codex agent, human supervision, bounded roles, task artifacts, and no autonomous merge/deploy/money-moving.

Sources

- Local reference: `/Users/chris/Desktop/crypto_prob_forecast/docs/gbc_prompts.md`
- Local reference: `/Users/chris/Desktop/crypto_prob_forecast/docs/CODEX_SUPERVISED_MULTI_AGENT_WORKFLOW.md`
- Local reference: `/Users/chris/Desktop/crypto_prob_forecast/docs/templates/CODEX_APPROVED_PLAN_TEMPLATE.md`
- OpenAI Codex cloud docs: https://platform.openai.com/docs/codex
- OpenAI Help Center, Codex CLI getting started: https://help.openai.com/en/articles/11096431-openai-codex-ci-getting-started
- OpenAI Cookbook, using `PLANS.md` for multi-hour Codex work: https://cookbook.openai.com/articles/codex_exec_plans/
- OpenAI, Harness engineering with Codex: https://openai.com/index/harness-engineering/
- GitHub Docs, Copilot CLI best practices: https://docs.github.com/en/copilot/how-tos/copilot-cli/cli-best-practices
- GitHub Docs, Copilot coding agent best practices: https://docs.github.com/en/enterprise-cloud@latest/copilot/tutorials/coding-agent/best-practices
- Anthropic Docs, Claude Code memory: https://docs.anthropic.com/en/docs/claude-code/memory